

TurboFlow 足球盘口技术方案 v2.0 设计与实施概述

1. 设计立场

1.1 业务约束

足球盘口业务有两个不同于 DEX 的核心特征：

1. **结果在确定时间点产生（终场哨响）**：所有结算判定都集中在比赛结束这个事件上，不存在 DEX 的连续撮合需求 这意味着结算可以批量、延迟容忍秒级、链上交易可以聚合
2. **赔率由平台主导而非市场博弈产生**：由后台运营维护赔率，引擎只做事件触发挂起；不需要 keeper 那种基于深度的赔率引擎

1.2 工程立场

维度	选择	取舍理由
资金真源	链上	安全合规要求、对账可审计
撮合形态	无	平台做庄，注单只需用户 → 平台合约单边交互
结算粒度	市场级批量	一笔链上交易结算一个市场所有注单，省 gas 又省状态机复杂度
链下镜像	必备	查询加速、风控前置、实时对账都依赖镜像
一致性策略	链上为真 + 三层兜底	扫块回流（被动）+ 实时对账（用户触发）+ 定时全量（兜底）
模块组织	按业务领域	DDD 思路，让代码结构反映业务而非技术架构

1.3 从 keeper 借鉴的三件事

- **链上为真源 + 链下镜像 + 扫块回流**：作为基础设施模式套用
- **两阶段状态机思路（业务态与执行态分离）**：本方案保留这一思路，但状态枚举重新设计
- **基础设施类**：Repositories 聚合、LRUCache、DelayQueue、ants 协程池、Redis 限流锁、Prometheus 指标接入

2. 业务闭环（用户视角）

完整一笔注单经历的环节，按时间顺序叙述：

1. **赛前**：运营在后台维护赔率与盘口可见性 系统把赔率推到客户端 WebSocket 通道
2. **下注**：用户选定某个选项（如 1x2 主胜），客户端发送 `PlaceBet` 请求 系统校验赔率与余额后，提交成功，在链下记录一张待上链注单，然后构造链上 `BetCreate` 交易并提交
提交链上失败，取消订单 给用户返回错误结果
3. **链上确认**：链上节点确认交易后产生 `BetCreated` 事件 链上域接模块扫到事件，把链下注单状态推进到 `Confirmed`，根据链上结果进行更新执行失败/成功，并把账户镜像中的 `frozen` 余额加上对应金额
4. **赛中**：赛事数据源推送进球等事件 系统按事件刷新「赛事实事」并触发市场短暂挂起；挂起期间禁止下注
5. **终场**：赛事数据源推送 `FullTime` 事件 结算服务对该比赛所有 `Open / Closed` 状态的市场逐个调用 `Evaluator` 判定每个选项的结果，异步逐条并发提交，由链上域接构造 `MarketSettle` 交易提交
6. **链上派奖**：合约根据传入的选项结果遍历该市场所有注单，命中赢家做派奖、命中 `push` 做退款，命中输家直接锁定 完成后产生一组 `BetSettled` 事件
7. **回流**：链上扫到对应链上事件，更新对应注单的业务态（`Won / Lost / Refunded`）、写入流水镜像，结束这笔注单的生命周期
8. **账户更新**：有专门监控链上钱包事件的程序，会去更新账户镜像(余额有变化会去触发用户对账)
9. **用户查询**：用户下次打开应用查询余额或注单时，账户域同步触发一次实时对账，确保看到的余额与链上一致

整个闭环没有订单簿、没有撮合，每笔注单都独立存在；结算靠合约批量执行，链下只承担判定 + 派发指令 + 镜像维护三件事

3. 系统架构（按业务领域划分）

3.1 服务进程外形

`turbo-soccer-book-service` 是一个 Go 单体服务，内部按业务域分包 **整个服务对外只暴露内网http（后台）与 WebSocket（赔率推送）**，不开放领域间 **RPC**——领域之间通过进程内接口调用与事件总线协作，避免分布式事务

3.2 业务领域划分

服务内部分为六个业务域加一组通用基础设施，关系图如下：

- 用户请求从外部进入 **API 接入层**，分发到对应领域：下注 / 撤注请求进 **注单域**；查询余额、查询注单进 **账户域**；查赔率、查盘口进 **盘口域**；后台运维接口进各域的 admin 接入点
- **赛事域** 单向消费外部赛事数据源，把归一化事件持久化并推到事件总线
- **盘口域** 订阅赛事域事件，做赔率发布、市场状态切换；也订阅赛事域的「终场」事件作为结算触发
- **注单域** 在下注路径上调用 **链上域** 提交交易；不直接写账户表，等待回流
- **结算域** 在收到「终场」事件后调用 Evaluator 判定结果，调用 **链上域** 提交结算交易
- **链上域** 是唯一可以提交链上交易和扫描链上事件的领域；扫到的事件分发给注单域、账户域、结算域做状态推进
- **账户域** 不直接被业务调用写余额——所有写入只能来源于链上域的事件回流；对外只提供读接口与实时对账能力
- **基础设施层** 提供配置、Redis、PGSQL、协程池、监控、日志、错误码、WebSocket 推送等共享能力

3.3 各领域内的组件

下面按领域列出 需要的核心组件：

3.3.1 赛事域 (match)

- **FeedClient**：连接外部数据源 WebSocket / HTTP，每家 Provider 一个 Client
- **EventNormalizer**：把不同 Provider 的协议归一化成内部 **MatchEvent**
- **EventStore**：写入 **match_event** 表（按 **seq_no** 单调递增，幂等）
- **FactAggregator**：增量维护 **match_fact** 表（仅 FT 维度的 **home_goals / away_goals / first_goal_team / first_goal_minute / last_goal_team / last_goal_minute**）
- **EventBusPublisher**：把关键事件（**Goal**、**Halftime**、**FullTime**）推到内部事件总线

3.3.2 盘口域 (market)

- **TemplateRegistry**：加载 **bet_market_template** 表，按 **code** 索引；启动时全量加载，运行时支持热更新
- **MarketLifecycle**：管理 **bet_market** 实例的 6 状态机（见第 6 节）
- **OddsBook**：缓存当前赔率，订阅运营后台变更与赛事事件触发的 **Suspended**
- **OddsPublisher**：把赔率变动推到 Redis Pub/Sub，由 WebSocket 网关下发给客户端

3.3.3 注单域 (bet)

- **BetService**：下注 / 撤注的领域入口
- **BetStateMachine**：业务态与链上态的双状态推进逻辑
- **BetRepository**：注单读写

- 注单域不直接写账户余额，全部走链上域事件回流

3.3.4 结算域 (settle)

- `EvaluatorRegistry` : 玩法族 Evaluator 注册中心
- `MarketSettler` : 消费「终场」事件，按市场调用 Evaluator，生成结算指令
- `SettleTaskQueue` : 结算指令队列（按市场加锁，串行提交链上）

3.3.5 账户域 (account)

- `AccountMirror` : 链下账户镜像（`account_balance` 表）的读接口
- `Ledger` : 流水镜像（`account_ledger` 表）
- `Reconciler` : 单用户实时对账的核心实现
- 账户域只做读 + 对账，写路径全部由链上域触发

3.3.6 链上域 (chain)

- `TxBuilder` : 根据领域请求构造链上指令（`BetCreate` / `BetCancel` / `MarketSettle` / `MarketVoid`）
- `TxSubmitter` : 管理链上 RPC 连接池、提交交易、维护 nonce / blockhash
- `TxTracker` : 跟踪已提交交易的确认状态
- `ChainScanner` : 扫描合约事件流，解码后分发给注单域、账户域、结算域
- `SlotCheckpoint` : 维护扫块位点，支持区间回放（补偿用）

3.3.7 基础设施 (common)

- `config` / `redis` / `db` / `monitor` / `websocket` / `event_bus`（进程内）/ `lock`（Redis 分布式锁）/ `pool`（ants 协程池）
- 这一层基本沿用 keeper 既有实现，不再重写

3.4 启动顺序

`main.go` 按以下顺序串行启动：基础设施初始化 → 领域服务初始化（赛事 → 盘口 → 账户 → 链上域 → 注单 → 结算）→ HTTP 服务器 → 后台 cron 先启动数据消费方再启动数据生产方，保证启动期事件不丢

4. 核心数据模型

字段命名遵循足球业务自描述原则，主键统一 bigint snowflake

4.1 赛事

```
1 match_league (id, name, country, season, logo, created_at, updated_at)
2
3 match_team (id, league_id, name, short_name, logo, created_at, updated_at)
4
5 match_info (
```

```

6   id,
7   league_id,
8   home_team_id, away_team_id,
9   kickoff_at,
10  venue, referee,
11  ext_match_id      varchar(64),    -- 数据源外部 ID
12  status            varchar(16),    -- Scheduled | Live | Halftime | E
13  created_at, updated_at,
14  unique(ext_match_id)
15 )
16
17 match_event (
18   id,
19   match_id,
20   type              varchar(16),    -- Phase 1: Goal | OwnGoal | Kicko
21   minute, added_minute,
22   team_id, player_id,
23   raw_payload       jsonb,
24   feed_seq          bigint,         -- 单调递增, 对账依据
25   feed_ts, ingested_at,
26   index(match_id, feed_seq)
27 )
28
29 match_fact (
30   match_id,
31   scope              varchar(8),    -- 固定 'FT'
32   home_goals, away_goals,
33   first_goal_team    varchar(8),    -- Home | Away | None
34   first_goal_minute,
35   last_goal_team     varchar(8),
36   last_goal_minute,
37   updated_at,
38   primary key (match_id, scope)
39 )
40

```

4.2 盘口

```

1  -- 盘口模板: ~25 条记录覆盖 35 盘口
2  bet_market_template (
3    id,
4    code              varchar(64) unique, -- e.g. '1x2_ft', 'total_ou_f
5    family             varchar(8),         -- A~H
6    name,
7    scope              varchar(8),         -- 固定 'FT'
8    data_source         varchar(16),       -- 固定 'goals'
9    ui_type             varchar(32),       -- ButtonGroup | OddsTable |
10   evaluator_code       varchar(64),       -- 指向 EvaluatorRegistry 注册
11   params_schema        jsonb,
12   settle_rules         jsonb,
13   phase               smallint,          -- 1
14   state               smallint           -- 1=enabled, 0=disabled
15 )
16
17 bet_market (
18   id,
19   match_id,
20   template_id,
21   params              jsonb,             -- 具体盘口参数: {line: 2.5, har
22   params_digest        varchar(64),       -- params 的稳定 hash, 用于去重
23   status               varchar(16),       -- Draft | Open | Suspended |
24   on_chain_addr        varchar(128),      -- 合约 PDA / 账户地址
25   open_at, suspend_at, close_at, settle_at,
26   settle_summary       jsonb,
27   unique(match_id, template_id, params_digest)
28 )

```

```

29
30 bet_selection (
31     id,
32     market_id,
33     option_key          varchar(64),          -- 'home' | 'draw' | 'away' |
34     label,
35     current_odds         decimal(10,4),
36     outcome              varchar(16),          -- Pending | Won | Lost | Pus
37     unique(market_id, option_key)
38 )
39
40 bet_odds_log (
41     id,
42     selection_id,
43     odds                 decimal(10,4),
44     source               varchar(16),          -- Trader | Engine | System
45     operator_id          bigint,              -- 运营 ID (Trader 来源)
46     created_at,
47     index(selection_id, created_at)
48 )
49

```

4.3 注单

```

1 bet_ticket (
2     id,
3     account_id,
4     selection_id,
5     market_id,
6     match_id,
7     stake               decimal(20,4),
8     stake_coin          varchar(16),
9     locked_odds         decimal(10,4),
10    potential_payout     decimal(20,4),
11    payout               decimal(20,4),        -- 实际派奖 (结算后填)
12    voucher_used_id      bigint,
13
14    biz_status           varchar(16),          -- Pending | Won | Lost | Refu
15    chain_status         varchar(16),          -- Pending | Submitted | Confi
16
17    bet_tx_hash          varchar(128),
18    cancel_tx_hash       varchar(128),
19    payout_tx_hash       varchar(128),
20    on_chain_addr        varchar(128),        -- 注单在合约里的账户地址 (PDA)
21
22    created_at, settled_at,
23    index(account_id, created_at),
24    index(market_id),
25    index(biz_status, chain_status)
26 )
27
28 bet_settle_task (
29     id,
30     market_id,
31     task_type           varchar(16),          -- MarketSettle | MarketVoid
32     payload             jsonb,               -- {selection_outcomes: [{opti
33     state               varchar(16),          -- Pending | Submitted | Confi
34     retry_cnt,
35     tx_hash             varchar(128),
36     locked_by, locked_at,
37     created_at, updated_at,
38     unique(market_id, task_type)
39 )
40

```

4.4 账户与链上交易

```
1 account_balance (  
2   account_id,  
3   coin          varchar(16),  
4   available     decimal(20,4),  
5   frozen        decimal(20,4),  
6   updated_at,  
7   primary key (account_id, coin)  
8 )  
9  
10 account_ledger (  
11   id,  
12   account_id,  
13   coin,  
14   action        varchar(32),      -- BetFreeze | BetUnfreeze | B  
15   amount         decimal(20,4),  
16   balance_after  decimal(20,4),  
17   ref_type       varchar(16),     -- Ticket | Settle | Recharge  
18   ref_id         bigint,  
19   tx_hash        varchar(128),  
20   remark,  
21   created_at,  
22   index(account_id, created_at),  
23   index(ref_type, ref_id)  
24 )  
25  
26 chain_tx (  
27   id,  
28   tx_hash        varchar(128) unique,  
29   tx_type        varchar(32),      -- BetCreate | BetCancel | Mar  
30   ref_type       varchar(16),     -- Ticket | SettleTask  
31   ref_id         bigint,  
32   status        varchar(16),      -- Submitted | Confirmed | Fai  
33   slot          bigint,  
34   err_msg,  
35   created_at, updated_at,  
36   index(ref_type, ref_id)  
37 )  
38  
39 chain_scan_checkpoint (  
40   scanner_name   varchar(32) primary key,  
41   last_slot      bigint,  
42   updated_at  
43 )  
44  
45 reconcile_log (  
46   id,  
47   account_id,  
48   trigger        varchar(16),      -- UserRead | EventReplay | Sc  
49   diff_summary   jsonb,  
50   fixed_summary  jsonb,  
51   status         varchar(16),      -- Ok | Fixed | Failed  
52   cost_ms,  
53   created_at,  
54   index(account_id, created_at)  
55 )  
56
```

写入纪律：

- `account_balance` 与 `account_ledger` 只能由链上域事件回流写入，业务流程任何时候都不直接 update 这一条是整个一致性方案的基石

- `bet_ticket.biz_status` 的 `Won/Lost/Refunded` 切换也只能由回流写入；业务流程只能写 `Pending → Cancelled`（在链上确认 `BetCancel` 之后）

5. 链上合约接口（业务面定义）

合约本身的实现由合约团队完成，本方案只锁定接口约定 需要的最小指令集与事件集如下

5.1 指令（链下 → 合约）

指令	入参	行为	触发事件
<code>MarketOpen</code>	<code>market_id</code> , <code>params</code> , <code>selections[]</code>	在合约内创建市场状态账户	<code>MarketOpened</code>
<code>BetCreate</code>	<code>market_id</code> , <code>selection_key</code> , <code>stake</code> , <code>locked_odds</code>	扣减用户 <code>available</code> 、 增加 <code>frozen</code> 、 生成注单账户	<code>BetCreated</code>
<code>BetCancel</code>	<code>bet_addr</code>	释放 <code>frozen</code> 回 <code>available</code> 、 关闭注单账户	<code>BetCancelled</code>
<code>MarketSettle</code>	<code>market_id</code> , <code>selection_outcomes[]</code>	合约内遍历该市场注单按结果批量派奖、退款、扣留	<code>BetSettled</code> × N
<code>MarketVoid</code>	<code>market_id</code>	合约内遍历该市场注单全部退款	<code>BetRefunded</code> × N

5.2 事件（合约 → 链下）

每个事件包含 `slot`、`tx_hash`、`market_id` 与该事件特定字段 链上域的 `ChainScanner` 解码后通过事件总线分发：

- `BetCreated` → 注单域：推进 `chain_status = Confirmed`、记录 `bet_tx_hash`；账户域：`available -= stake`、`frozen += stake`、写一条 `BetFreeze` 流水
- `BetCancelled` → 注单域：`biz_status = Cancelled`；账户域：`frozen -= stake`、`available += stake`、写一条 `BetUnfreeze` 流水

- `BetSettled` → 注单域：按 outcome 推进 `biz_status` 为 `Won/Lost/Refunded`；账户域：根据 outcome 写 `BetPayout` / `BetRefund` 流水并调整余额
- `BetRefunded` → 与 `BetSettled` 的 `Refunded` 分支相同

5.3 设计要点

- **市场级批量派奖**：`MarketSettle` 一次交易内派完该市场所有注单，避免链下逐单提交带来的状态机膨胀 这是与 keeper（订单粒度执行）最大的差异
- **注单 PDA**：每张注单在合约内有独立 PDA 账户（`on_chain_addr`），便于 `BetCancel` 精确定位
- **幂等**：合约对相同 `market_id` 的 `MarketSettle` 只接受一次（重复提交直接拒绝），保证链下重试不会双重派奖

6. 关键流程

6.1 市场状态机

当前态	目标态	触发
Draft	Open	开盘时间到达或运营手动开盘（同时上链 <code>MarketOpen</code> ）
Open	Suspended	进球事件
Suspended	Open	挂起时长到（默认 30 秒）
Open / Suspended	Closed	比赛终场
Closed	Settled	<code>MarketSettle</code> 链上确认 + 该市场所有 <code>BetSettled</code> 事件回流完成
任意非 Settled	Voided	触发 <code>FR027 void</code> 条件后 <code>MarketVoid</code> 链上确认

6.2 注单状态机

业务态 `biz_status`（5 种）与链上态 `chain_status`（4 种）独立推进：

chain_status	含义
Pending	链下草稿已写，链上交易未提交或提交中
Submitted	链上交易已发出，等待确认
Confirmed	链上确认 + 事件回流完成
Failed	提交失败或长时间未确认，进对账

biz_status	含义
Pending	受理中（含已上链等结算）
Won	中奖且派奖事件已回流
Lost	未中奖
Refunded	退款（push / void）
Cancelled	撤注成功

不引入 keeper 的 CommitMiss 中间态：所有链上交易状态不明确统一走 Failed → 对账中心

6.3 下注流程

按时间顺序：

1. 客户端调用 POST /bet/place，传 selection_id / stake / locked_odds
2. 注单域校验：参数完整性、市场为 Open、locked_odds 与当前 bet_selection.current_odds CAS 一致 下单风控这块的设计 规则引擎(下注的开始-结束时间，下注金额的限制（不同的时间下注金额不一样,可配置），市场的开放时间，当前的赔率<? 只允许下注N球队，按照赔率对下注金额有限制 平台限制 最大持仓额度限制 每一个市场N球队都有最大持仓限制)
3. 调账户域 Reconciler 做预校验级对账（仅检查镜像不为负 + 镜像 available >= stake）这不是授权校验，只是排掉明显异常的请求
4. 注单域写一张草稿注单：biz_status = Pending、chain_status = Pending
5. 注单域调用链上域 TxBuilder.BuildBetCreate，再调 TxSubmitter.Submit 提交成功更新 chain_status = Submitted、bet_tx_hash；失败更新 chain_status = Failed 并通知客户端

6. 立即给客户端返回 `{ ticket_id, biz_status: Pending, chain_status: Submitted }`
7. 链上确认后 `ChainScanner` 解码 `BetCreated` 事件，注单域更新 `chain_status = Confirmed`、账户域写 `BetFreeze` 流水并调整镜像余额
8. 整个下注流程结束 客户端可以主动轮询或订阅注单变更通道得到 `Confirmed` 状态

任何一步失败的回滚都不依赖反向链上交易：草稿注单只在 `chain_status = Confirmed` 后才会被对账中心当作真实存在的资金占用处理；提交失败的草稿在 24 小时无确认时由 cron 清理

6.4 撤注流程

仅在 `bet_market.status = Open`、且当前时间未超过 `system_config.bet_cancel_window_sec` 时允许：

1. 注单域校验后调链上域提交 `BetCancel`
2. 链上确认后，回流事件把注单 `biz_status = Cancelled`、账户域写 `BetUnfreeze` 流水

6.5 结算流程

由终场事件触发，是整个系统最复杂也最关键的流程：

判定阶段（结算域）：

1. 赛事域投递 `FullTime` 事件到事件总线
2. 结算域监听到事件，拉出该 `match_id` 下所有 `bet_market.status ∈ {Open, Closed}` 的市场
3. 对每个市场，先把状态改为 `Closed`（保证不再有新下注被合约接受）
4. 读取最新的 `match_fact` 行（FT）
5. 通过 `template.evaluator_code` 在 `EvaluatorRegistry` 中找到 `Evaluator` 实现
6. 对该市场下每个 `bet_selection` 调用 `Evaluator.Evaluate(selection, fact, params)`，得到 `Outcome ∈ {Won, Lost, Push, Void, HalfWin, HalfLose}`
7. 把所有选项的结果异步并发发送
8. 把任务加入提交队列

提交阶段（链上域）：

1. 提交器以 `market_id` 为分布式锁串行处理（Redis 锁，TTL 60s）
2. 构造 `MarketSettle` 链上交易并提交，`task.state = Submitted`，写 `chain_tx`
3. 链上确认后，`task.state = Confirmed` 提交失败按指数退避重试，重试上限耗尽进 `Failed` 并告警

回流阶段（链上域 → 注单 / 账户 / 盘口）：

- 1. ChainScanner 扫到 BetSettled 事件流，按事件中的 bet_addr 反查注单 ID
- 2. 注单域：按事件 outcome 把 biz_status 推进到 Won / Lost / Refunded，记录 payout 与 payout_tx_hash
- 3. 账户域：写对应流水（BetPayout / BetRefund / 输单只解冻不入账即写 BetFreeze 反向流水），更新 account_balance 镜像
- 4. 当某市场的所有注单回流完毕，盘口域把 bet_market.status = Settled、写 settle_summary
- 5. 通知服务给中奖用户推送站内信

整体并发控制：

- 同一市场的结算交易串行（市场级锁）
- 不同市场可并行，由 ants 协程池控制吞吐
- 单笔 MarketSettle 内合约自身保证遍历该市场注单的原子性，链下无须再做注单级锁

7. 结算引擎：玩法族 Evaluator

7.1 接口

```
1 type Outcome string
2 const (
3     OutcomeWon      Outcome = Won
4     OutcomeLost     Outcome = Lost
5     OutcomePush     Outcome = Push      // 整数盘口线退款
6     OutcomeVoid     Outcome = Void      // 数据缺失或合规判定
7     OutcomeHalfWin  Outcome = HalfWin   // 亚洲盘 1/4 球
8     OutcomeHalfLose Outcome = HalfLose
9 )
10
11 type Evaluator interface {
12     Code() string                // 在 Registry 中的注册名
13     Evaluate(
14         sel    *Selection,
15         fact   *MatchFact,
16         params map[string]any,
17     ) (Outcome, error)
18 }
19
```

7.2 注册的 8 个 Evaluator

Code	实现	覆盖盘口
result	ResultEvaluator	1x2、平局返还、双胜彩
over_under	OverUnderEvaluator	合计 O/U、主队合计、客队合计、亚洲总进球

		O/U、两个半场高于 / 低于 1.5
handicap	HandicapEvaluator	亚洲让分盘、欧洲让球
boolean	BooleanEvaluator	BTTS、主队不失球、客队不失球、主队不丢球获胜、客队不丢球获胜
exact_match	ExactMatchEvaluator	正确进球 5×5、总进球数、多进球、进球范围、主队 / 客队准确进球、主队 / 客队多进球、Multiscores、输赢比数
combined	CombinedEvaluator	1x2&BTTS、1x2&合计、双胜彩&合计、合计&BTTS
parity	ParityEvaluator	总进球奇 / 偶、主队奇 / 偶、客队奇 / 偶
first_last	FirstLastEvaluator	谁先进 1st 球、最后一个进球属于

合计覆盖 34 个盘口

7.3 配置驱动复用示例

OverUnderEvaluator 一份实现服务 5 个 盘口：

盘口	template.params
全场总进球 O/U	{ data_field: goals_total, line: 2.5 }
主队合计	{ data_field: goals_home, line: 1.5 }
客队合计	{ data_field: goals_away, line: 1.5 }

亚洲总进球 O/U	{ data_field: goals_total, line: 2.25 }
两个半场高于 / 低于 1.5	{ data_field: goals_total, line: 1.5 }

`fact.ExtractField(params[data_field])` 拿到数值，再与 `params[line]` 比较，按整数 / 半整数 / 四分之一规则输出 Outcome Phase 2 加半场 / 角球 / 罚牌时只增加 `data_field` 的支持值，不改 Evaluator 本身

7.4 Evaluator 测试策略

每个 Evaluator 配套表驱动测试，覆盖：

- 边界值（0:0、整数线打平、半整数线、四分之一线两侧）
- Void 场景（fact 缺失字段时返回 `Void` 而非崩溃）
- 大小写 / option_key 拼写一致性

测试用例集本身就是 PRD 第 9 章 FR001-FR012 结算规则的可执行版本

8. 实时对账（单用户）

8.1 设计目标

- 用户每次读余额、读注单时，确保看到的镜像与链上一致
- 不阻塞写路径（下注 / 撤注），写路径始终以链上裁决为准
- 失败可降级，链上 RPC 失败时退化为链下自洽校验
- 单次开销可控， $P95 \leq 200ms$

8.2 触发场景

- 用户登录后第一次拉取余额或注单列表
- 用户提现 / 充值前的余额查询
- 链上域扫到该用户相关的 `BetSettled` / `BetRefunded` 事件、即将更新镜像之前的最后校验
- 后台运营手动触发

8.3 三组等式校验

`Reconciler.Reconcile(account_id)` 执行三组校验：

1. 链上 = 镜像：链上 `available + frozen` 与 `account_balance.available + frozen` 是否相等

2. 冻结额 = 未结注单冻结总和： `account_balance.frozen` 与 `bet_ticket.biz_status = Pending && chain_status = Confirmed` 的 stake 总和（按币种分组）是否相等
3. 流水净额 = 余额变化： `account_ledger` 净额与镜像余额关系（仅在差异修复路径启用，不每次都跑）

三组全部成立返回 `Ok`；任一失败进入修复

8.4 修复策略

1. 用 Redis 锁 `reconcile:{account_id}`（TTL 10 秒）保证同账户串行修复
2. 重新拉取链上账户与最近一段时间的链上事件
3. 用链上数据覆盖 `account_balance` 镜像
4. 扫描该账户的 `bet_ticket`，把链上已确认但本地仍是 `Submitted` 的注单按 `tx_hash` 主动追平 `chain_status = Confirmed`，并补写流水
5. 写 `reconcile_log`：`status = Fixed`、`diff_summary`、`fixed_summary`、`cost_ms`
6. 连续 3 次修复后差异仍存在的，告警 + 冻结该用户下注接口直至人工介入

8.5 性能与缓存

- 成功对账后写 Redis `reconcile:ok:{account_id}`，TTL 30 秒；同一用户 30 秒内再次触发直接命中缓存返回 `Ok`
- 缓存策略保证赛事密集时段不会被链上 RPC 风暴打挂
- 对账失败不进缓存

8.6 与定时全量对账的边界

- 实时对账只覆盖被读到的活跃用户
- 定时全量对账（每小时一次的冻结一致性校验）覆盖全量长尾用户与冷账户
- 两者独立运行，互为兜底

9. 异常补偿

9.1 补偿矩阵

场景	一级（自动）	二级（半自动）	三级（人工）
Feed 漏传单条	Provider 重推	<code>CompensateService.ReplayFeedRange</code> 按	Admin 后台手动补录

		feed_seq 区间 回放	
链上事件漏扫	扫描器 slot 缓冲 重扫	CompensateSe rvice.Replay ChainRange 按 slot 区间回放	Admin 按 tx_hash 单笔 补偿
下注 / 撤注交易 失败	指数退避重试	重试耗尽后通知 客户端 + 保留草 稿	客户重发
结算交易失败	指数退避重试	任务进 Failed + 告警	后台手动重试结 算
用户余额异常	实时对账触发覆 盖	定时冻结一致性 扫描	后台手动同步链 上 → 镜像
FR027 VAR void	结算域生成反向 MarketVoid 任 务	链上反向交易 + 镜像回退	合规审核

9.2 定时任务清单

由 ScheduleService 注册：

Cron	任务	目的
每 30 秒	扫 chain_scan_checkp oint slot 空洞	链上事件不漏扫
每 30 秒	扫 feed 消费位点空洞	赛事情件不漏
每 5 分钟	扫超时未结算的市场	终场后 5 分钟仍未 Settled 即告警
每 5 分钟	扫 bet_settle_task.s tate = Failed	结算交易失败兜底
每 1 小时	全量冻结一致性校验	验证 account_balance.f

		rozen 总和 = Pending 注单冻结总和
每天 00:15	归档冷数据	match_event / match_fact 超期数 据迁出

10. 高可用与部署

单节点起步：

- 单进程部署 turbo-soccer-book-service
- 链上域的 ChainScanner 可独立成 Pod 部署（避免重启时影响主服务）
- 数据库主从、Redis Sentinel 沿用现有基础设施
- 故障切换：进程重启后从 chain_scan_checkpoint 与 feed 消费位点续扫，30 秒内恢复

Phase 2 数据量上来后再做按 league_id 的水平分片，方案与 keeper docs/ha-design/ 一致

11. 实施里程碑

里程碑	交付内容	关键产出
M1.1 基础设施	项目骨架、配置、日志、监控、DB / Redis 连接、错误码	可启动的空服务
M1.2 合约对接	与合约团队冻结 IDL；用 Stub 合约打通端到端	BetCreate / BetCancel / MarketSettle 通路
M1.3 赛事域	Feed 接入、match_event / match_fact 落库、事件总线	进球事件能驱动盘口域
M1.4 盘口域	25 条 template 数据迁移、运营后台改赔率、WebSocket 推送	客户端可见赔率
M1.5 注单域	PlaceBet / CancelBet 完整闭环 + 状态机	端到端可下注

M1.6 账户域 + 实时对账	镜像表、流水、Reconciler 三组等式 + 修复	用户读余额触发对账
M1.7 结算域 + 8 个 Evaluator	结算判定、MarketSettle 提交、回流派奖	一场比赛能完整跑完
M1.8 补偿与对账	链上 / feed 双通道补偿、定时 全量校验	异常场景兜底
M1.9 监控与冒烟	Prometheus 指标、告警、E2E 自动化用例	上线就绪

验收口径：

- 34 盘口全流程闭环（下注 / 撤注 / 结算派奖均经链上确认）
- 结算准确率 $\geq 99.9\%$
- 下注响应 $P95 \leq 500\text{ ms}$ （含链上提交）
- 赔率推送延迟 $P95 \leq 2\text{ s}$
- 单用户实时对账 $P95 \leq 200\text{ ms}$ ；首次对账成功率 $\geq 99.5\%$
- 7 天连续运行不出现累积差异

12. 后续阶段索引

为保持 文档聚焦，后续阶段只在此处登记：

阶段	主要新增	不需要重构的部分
Phase 2	半场聚合（HT1 / HT2）、半场盘口模板、角球 / 罚牌事件流、自动赔率引擎	Evaluator 接口、市场状态机、链上合约接口、对账中心
Phase 3	球员级事件流与 fact、GoalscorerEvaluator / PlayerStatEvaluator / TimeWindowEvaluator、SGM 同场复式、合约新增	上述全部

	BetMultiCreate 指令	
--	-------------------	--

的所有抽象（Evaluator 接口、scope / data_source 配置位、市场状态机、链上指令命名规则）都已经为 Phase 2/3 留好扩展点，后续不需要重构